UNIVERSIDADE FEDERAL DO CEARÁ

TECHNICAL REPORT

Aluno: Glauber De Melo Duarte e Emily Hannah Eufrasio Bastos

Introdução

Na avaliação foram utilizados dois datasets fornecidos pelo professor da plataforma kaggle no repositório da turma no github, os datasets foram um de classificação e o outro de regressão.

Classificação: ***Drug Classification***

O dataset de classificação utilizado foi o Drug Classification, obtido na plataforma kaggle, ele contém informações sobre medicamentos prescritos com base em características demográficas e clínicas dos pacientes, como idade, pressão arterial e nível de colesterol. O objetivo é prever a classe do medicamento mais adequado para um paciente com base nessas informações, utilizando técnicas de aprendizado de máquina para classificação.

Características Relevantes:

- ***Idade: Faixa etária do paciente.***
- ***Pressão Arterial (BP): Indicada como baixa, normal ou alta.***
- ***Colesterol: Classificado como normal ou alto.***
- ***Nível de Sódio no Sangue (Na_to_K): Valor numérico relacionado à proporção de sódio e potássio no sangue.***
- ***Classe do Medicamento: O atributo alvo com cinco categorias possíveis.***

Regressão (Laptop Prices Dataset)
O dataset para regressão também extraído do kaggle, contém informações detalhadas sobre laptops disponíveis no mercado, incluindo características técnicas, preço e avaliação dos usuários. O objetivo principal é prever o preço dos laptops com base em suas especificações técnicas e popularidade.

Características Relevantes:

- ***Marca****: Fabricante do laptop.*
- ***Especificações Técnicas****: Processador, geração do processador, quantidade de RAM, armazenamento (SSD/HDD), e presença de GPU.*
- ***Avaliação dos Usuários****: Número de avaliações e média de avaliação.*

- ***Preço**: O atributo alvo que se busca prever.*

2. Observações

Limpeza e Pré-processamento dos Dados

Questões Relacionadas: 5 e 6

O que Aconteceu:
*Quando começamos a trabalhar com o dataset de laptops, percebemos que algumas colunas tinham informações confusas ou inconsistentes. Por exemplo, na coluna **weight**, havia valores como "ThinNlight" misturados com números. Tivemos que gastar um tempo extra para extrair só os números. Além disso, na coluna **rating**, as avaliações estavam no formato "5 stars", e precisamos limpar esses dados para que fossem interpretados como números. Foi um pouco trabalhoso, mas isso me ajudou a entender a importância de uma boa preparação dos dados.*

Escolher o Atributo Mais Relevante e Normalizar os Dados

- ***Questões Relacionadas:** Questões 1, 5, 6 e 7*
- ***O que Aconteceu:***
  *Foi muito complicado escolher o atributo que mais influenciava o preço no dataset. Depois de calcular a correlação, vimos que o atributo **Number of Ratings** era o mais relevante, mas, mesmo assim, a relação não era muito forte. Além disso, percebemos que os dados precisavam ser normalizados para melhorar os resultados, o que fez o processo demorar mais um pouco.*
- ***Observação:** Interpretar o dataset **Drug Classification** foi bem mais tranquilo porém demorou um pouco pra entender que na verdade todos os atributos eram os mais relevantes pois se tratava de medicamentos e qualquer informação sobre saúde importa, logo percebemos que não haveria colunas irrelevantes para se remover e criar um novo dataframe*

3. Resultados e discussão

Questão 1: Manipulação e Pré-processamento de Dados

Fluxograma do Processo

Importação e Análise Inicial:

a. Importar o dataset utilizando ***load drug dataset.***
b. Verificar a presença de valores ausentes (NaN).

Tratamento de Valores Ausentes:

c. Remover as linhas contendo valores ausentes, se existirem, para obter um dataset limpo.

Seleção de Colunas Relevantes:

d. Identificar as colunas mais importantes para a análise (**Age, Sex, BP, Cholesterol, Na_to_K, Drug**).

Análise da Distribuição de Classes:

e. Observar a distribuição das classes na coluna **Drug**.

Conversão de Classes:

f. Converter a coluna Drug para valores numéricos, facilitando futuras análises e algoritmos.

Salvar o Dataset Ajustado:

g. Salvar o dataset final como drug_dataset_ajustado.csv.

Implementação

```python
# Carregar o dataset
from src.utils import load_drug_dataset

drug = load_drug_dataset()

if drug.isnull().sum().sum() > 0:
    print("Existem valores NaN. Removendo...")
    drug_cleaned = drug.dropna()
else:
    print("Não há valores NaN.")
    drug_cleaned = drug.copy()

print("Colunas disponíveis:", drug_cleaned.columns)

columns_to_keep = ['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K', 'Drug']
drug_filtered = drug_cleaned[columns_to_keep]

print("\nDistribuição de classes:")
print(drug_filtered['Drug'].value_counts())

drug_filtered['Drug'] = drug_filtered['Drug'].astype('category').cat.codes
print("\nClasses convertidas para valores numéricos.")


# Exibir o dataframe final
print("\nDataframe final:\n", drug_filtered.head())

# Salvar o dataset atualizado
drug_filtered.to_csv("drug_dataset_ajustado.csv", index=False)
print("\nDataset ajustado salvo como 'drug_dataset_ajustado.csv'.")
```

Resultados

```
C:\Users\glaub\AppData\Local\Microsoft\WindowsApps\python3.12.exe C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\questao1.py
Não há valores NaN.
Colunas disponíveis: Index(['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K', 'Drug'], dtype='object')

Distribuição de classes:
Drug
DrugY    91
drugX    54
drugA    23
drugC    16
drugB    16
Name: count, dtype: int64

Classes convertidas para valores numéricos.

Dataframe final:
    Age Sex      BP Cholesterol  Na_to_K  Drug
0   23   F    HIGH        HIGH   25.355     0
1   47   M     LOW        HIGH   13.093     3
2   47   M     LOW        HIGH   10.114     3
3   28   F  NORMAL        HIGH    7.798     4
4   61   F     LOW        HIGH   18.043     0

Dataset ajustado salvo como 'drug_dataset_ajustado.csv'.
```

Podemos notar que nosso dataset não possuía valores NaN e que todas as colunas disponíveis eram relevantes, onde decidimos não remover nenhuma, por causa do contexto do dataset Drug Classification.

Questão 2: Classificação Utilizando KNN Manual

Preparação dos Dados:

a. Importar o dataset atualizado (drug_dataset_ajustado.csv).
b. Separar os atributos (**X**) e o alvo (y) para classificação.
c. Dividir o dataset em conjuntos de treino e teste, sem normalização.

Implementação do KNN Manual:

d. Implementar uma função para calcular distâncias baseadas em quatro métricas:
    - Distância de Mahalanobis.
    - Distância de Chebyshev.
    - Distância de Manhattan.
    - Distância Euclidiana.
e. Utilizar o valor k = 7 para determinar os vizinhos mais próximos.

Avaliação do Modelo:

f. Calcular a acurácia do KNN para cada métrica de distância.
g. Comparar os resultados e determinar qual métrica apresentou o melhor desempenho.

Implementação

```python
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
from scipy.spatial.distance import mahalanobis, chebyshev, cityblock, euclidean


def calculate_distance(metric, point1, point2, cov_matrix=None):  1 usage  devgluber
    if metric == "mahalanobis":
        return mahalanobis(point1, point2, cov_matrix)
    elif metric == "chebyshev":
        return chebyshev(point1, point2)
    elif metric == "manhattan":
        return cityblock(point1, point2)
    elif metric == "euclidean":
        return euclidean(point1, point2)
    else:
        raise ValueError("Métrica desconhecida.")

def knn(X_train, y_train, X_test, k, metric):  devgluber
    predictions = []
    cov_matrix_inv = None
    X_train_array = np.array(X_train, dtype=np.float64)  # Garante conversão numérica

    if metric == "mahalanobis":
        cov_matrix = np.cov(X_train_array.T)
        cov_matrix_inv = np.linalg.inv(cov_matrix)

    for test_point in X_test:
        distances = []
        for i, train_point in enumerate(X_train_array):
            dist = calculate_distance(metric, test_point, train_point, cov_matrix_inv)
            distances.append((dist, y_train[i]))

        distances = sorted(distances, key=lambda x: x[0])[:k]
        neighbors_classes = [d[1] for d in distances]
        predicted_class = max(set(neighbors_classes), key=neighbors_classes.count)
        predictions.append(predicted_class)

    return np.array(predictions)

def evaluate_knn(X_train, y_train, X_test, y_test, k, metric):  1 usage  devgluber
    X_train_array = X_train.to_numpy(dtype=np.float64)
```

```python
    return np.array(predictions)

def evaluate_knn(X_train, y_train, X_test, y_test, k, metric):  1 usage  devgluber
    X_train_array = X_train.to_numpy(dtype=np.float64)
    X_test_array = X_test.to_numpy(dtype=np.float64)
    y_train_array = y_train.to_numpy()
    y_test_array = y_test.to_numpy()

    y_pred = knn(X_train_array, y_train_array, X_test_array, k, metric)
    accuracy = np.mean(y_pred == y_test_array)
    return accuracy


dataset_path = r'C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\Datasets\drug_dataset_ajustado.csv'
drug_data = pd.read_csv(dataset_path)

X = drug_data.drop(columns=['Drug'])
y = drug_data['Drug']

X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

metrics = ["mahalanobis", "chebyshev", "manhattan", "euclidean"]
k = 7
results = {}

for metric in metrics:
    acc = evaluate_knn(X_train, y_train, X_test, y_test, k, metric)
    results[metric] = acc
    print(f"Acurácia usando {metric}: {acc:.2f}")
```

Questão 3: Impacto da Normalização na Classificação com KNN

Fluxograma do Processo

- **Preparação do Dataset:**
  - Carregar o dataset atualizado (drug_dataset_ajustado.csv).
  - Separar as colunas de atributos (X) e rótulos (y).
  - Dividir os dados em conjuntos de treino e teste.
- **Aplicação de Normatizações:**
  - Normalização Logarítmica: Aplicar log1p para transformar os dados, reduzindo o impacto de valores extremos.
  - Normalização de Média Zero e Variância Unitária: Utilizar StandardScaler para centralizar os dados em torno de zero com desvio padrão unitário.
- **Execução do KNN**:
  - Utilizar o KNN manual com a métrica de distância Euclidiana (escolhida com base nos resultados da Questão 2).
  - Avaliar a acurácia do modelo para cada tipo de normalização.
- **Comparação de Resultados:**
  - Imprimir as acurácias obtidas lado a lado para comparar o impacto de cada normalização.

Implementação

```python
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


def knn(X_train, y_train, X_test, k, metric):  devgluber
    """
    Implementação manual do KNN com diferentes métricas.

    Parâmetros:
        X_train: Dados de treino (features)
        y_train: Rótulos de treino
        X_test: Dados de teste (features)
        k: Número de vizinhos
        metric: Métrica de distância ('euclidean', 'manhattan', 'chebyshev', 'mahalanobis')

    Retorna:
        y_pred: Predições para os dados de teste
    """
    y_pred = []

    if metric == 'mahalanobis':
        # Calcula a matriz de covariância
        cov_matrix = np.cov(X_train, rowvar=False)
        inv_cov_matrix = np.linalg.inv(cov_matrix)
    else:
        inv_cov_matrix = None

    for x_test in X_test:
        if metric == 'mahalanobis':
            # Calculando a distância de Mahalanobis
            distances = np.array(
                [np.sqrt(np.dot(np.dot((x_train - x_test), inv_cov_matrix), (x_train - x_test).T)) for x_train in
                 X_train])
        else:
            # Para outras métricas, usa-se o cdist com a métrica especificada
            distances = cdist( XA: [x_test], X_train, metric=metric)[0]

        # Obtém os índices dos k vizinhos mais próximos
```

```python
def knn(X_train, y_train, X_test, k, metric):  devgluber

        # Obtém os índices dos k vizinhos mais próximos
        neighbors_idx = np.argsort(distances)[:k]

        # Obtém os rótulos dos k vizinhos
        neighbor_labels = y_train[neighbors_idx]

        # Vota pela maioria
        most_common = Counter(neighbor_labels).most_common(1)[0][0]
        y_pred.append(most_common)

    return np.array(y_pred)


def evaluate_knn(X_train, y_train, X_test, y_test, k, metric):  2 usages  devgluber
    """
    Avaliação do KNN com base em uma métrica de distância.

    Parâmetros:
        X_train: Dados de treino (features)
        y_train: Rótulos de treino
        X_test: Dados de teste (features)
        y_test: Rótulos reais para os dados de teste
        k: Número de vizinhos
        metric: Métrica de distância ('euclidean', 'manhattan', 'chebyshev', 'mahalanobis')

    Retorna:
        Acurácia do modelo nos dados de teste
    """
    y_pred = knn(X_train, y_train, X_test, k, metric)
    accuracy = np.mean(y_pred == y_test)
    return accuracy


# Carregando o dataset de drogas
dataset_path = r'C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\Datasets\drug_dataset_ajustado.csv'
drug_data = pd.read_csv(dataset_path)

# Dividindo as colunas de features e rótulos
X = drug_data.drop(columns=['Drug'])  # Droppando a coluna 'Drug' para obter as features
y = drug_data['Drug']  # A coluna 'Drug' será o rótulo
```

```python
# Dividindo os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.3, random_state=42)

# Normalização Logarítmica
X_train_log = np.log1p(X_train)
X_test_log = np.log1p(X_test)

# Normalização de Média Zero e Variância Unitária
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Definindo o número de vizinhos
k = 7
metric = 'euclidean'  # Definindo a métrica de distância

# Avaliando o KNN para os dados normalizados logaritmicamente
acc_log = evaluate_knn(X_train_log.to_numpy(), y_train.to_numpy(), X_test_log.to_numpy(), y_test.to_numpy(), k, metric)

# Avaliando o KNN para os dados normalizados com média zero e variância unitária
acc_scaled = evaluate_knn(X_train_scaled, y_train.to_numpy(), X_test_scaled, y_test.to_numpy(), k, metric)

# Imprimindo as acurácias lado a lado
print(f"Acurácia com Normalização Logarítmica: {acc_log:.2f}")
print(f"Acurácia com Normalização de Média Zero e Variância Unitária: {acc_scaled:.2f}")
```

Resultados

```
C:\Users\glaub\AppData\Local\Microsoft\WindowsApps\python3.12.exe C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\questao3.py
Acurácia com Normalização Logarítmica: 0.92
Acurácia com Normalização de Média Zero e Variância Unitária: 0.85
```

Questão 4: Busca pela Melhor Parametrização do KNN

1. Carregamento e Pré-processamento dos Dados:

- O conjunto de dados foi carregado a partir de um arquivo CSV. As colunas de características foram separadas da coluna de rótulos (target).
- A divisão dos dados em conjuntos de treino e teste foi realizada, utilizando 70% dos dados para treino e 30% para teste.
- A normalização dos dados foi feita com a técnica de *StandardScaler* , que aplica normalização com média zero e variância unitária. Esse método foi escolhido com base nos bons resultados obtidos em uma etapa anterior do projeto.

2. Implementação do KNN:

- A implementação do KNN foi feita manualmente, permitindo o uso de diferentes métricas de distância (Euclidiana, Manhattan, Chebyshev e Mahalanobis). O código usa a função **cdist** para calcular as distâncias entre os dados de teste e treino e escolhe os vizinhos mais próximos para fazer a classificação.

- A distância de Mahalanobis foi separada separadamente devido à necessidade de calcular uma matriz de covariância e sua inversão.

3. Avaliação do Modelo:

- A função **evaluate_knn** avalia o desempenho do modelo, calculando a acurácia para diferentes valores de k. A acurácia é uma proporção de correção correta em relação ao total de correção feita.
- Para isso, foram testados valores de k de 1 a 15 e, para cada valor, a acurácia foi calculada e armazenada.

Fluxograma do Processo:

1. **Carregamento do Dataset:** Carregar os dados de um arquivo CSV.
2. **Pré-processamento:** Separar as características e rótulos, e dividir os dados em treino e teste.
3. **Normalização:** Aplique a normalização (StandardScaler) nos dados de treino e teste.
4. **Execução do KNN:** Para cada valor de k (de 1 a 15), calcule as distâncias usando uma métrica selecionada e determine a classe mais frequente entre os k vizinhos mais próximos.
5. **Avaliação:** Calcule a acurácia para cada valor de registro dos resultados.
6. **Visualização:** Plotar um gráfico com as acurácias obtidas para os diferentes valores de e identificar o melhor valor.

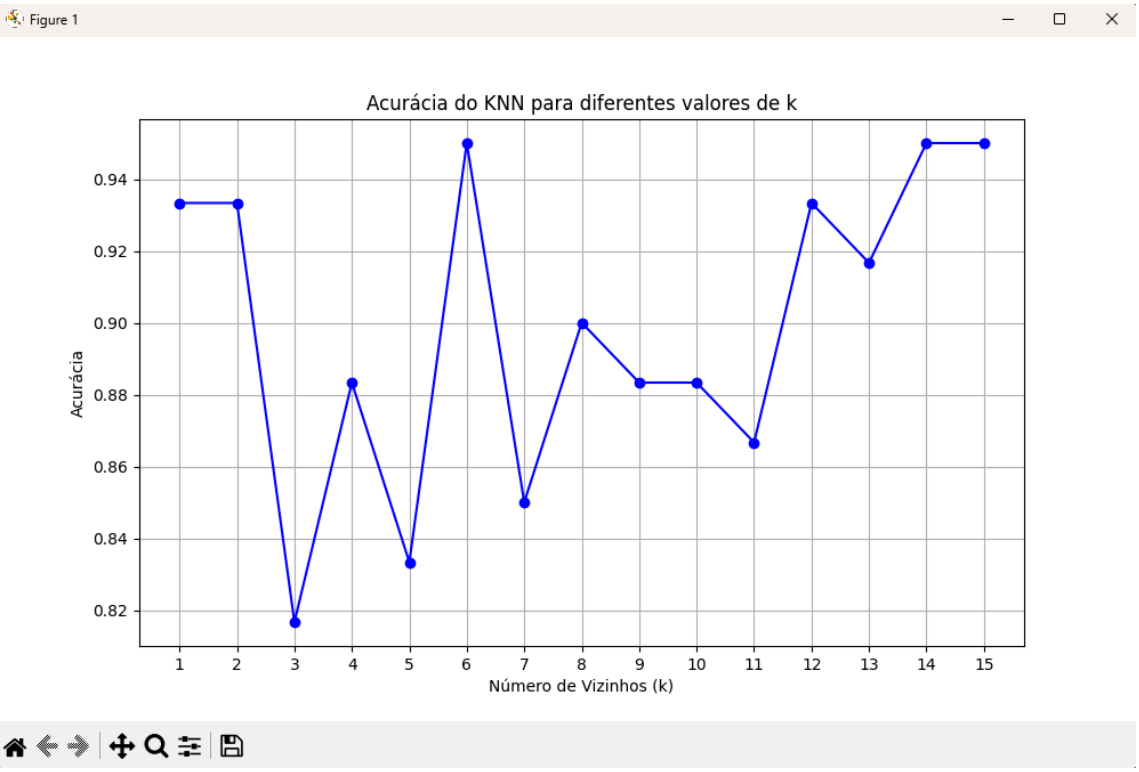
Implementação

```python
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt


def knn(X_train, y_train, X_test, k, metric):  devgluber
    """
    Implementação manual do KNN com diferentes métricas.

    Parâmetros:
        X_train: Dados de treino (features)
        y_train: Rótulos de treino
        X_test: Dados de teste (features)
        k: Número de vizinhos
        metric: Métrica de distância ('euclidean', 'manhattan', 'chebyshev', 'mahalanobis')

    Retorna:
        y_pred: Predições para os dados de teste
    """
    y_pred = []

    if metric == 'mahalanobis':
        # Calcula a matriz de covariância
        cov_matrix = np.cov(X_train, rowvar=False)
        inv_cov_matrix = np.linalg.inv(cov_matrix)
    else:
        inv_cov_matrix = None

    for x_test in X_test:
        if metric == 'mahalanobis':
            # Calculando a distância de Mahalanobis
            distances = np.array(
                [np.sqrt(np.dot(np.dot((x_train - x_test), inv_cov_matrix), (x_train - x_test).T)) for x_train in
                 X_train])
        else:
            # Para outras métricas, usa-se o cdist com a métrica especificada
            distances = cdist( XA: [x_test], X_train, metric=metric)[0]
```

```python
            # Para outras métricas, usa-se o cdist com a métrica especificada
            distances = cdist( XA: [x_test], X_train, metric=metric)[0]

        # Obtém os índices dos k vizinhos mais próximos
        neighbors_idx = np.argsort(distances)[:k]

        # Obtém os rótulos dos k vizinhos
        neighbor_labels = y_train[neighbors_idx]

        # Vota pela maioria
        most_common = Counter(neighbor_labels).most_common(1)[0][0]
        y_pred.append(most_common)

    return np.array(y_pred)


def evaluate_knn(X_train, y_train, X_test, y_test, k, metric):  1 usage  devgluber
    """
    Avaliação do KNN com base em uma métrica de distância.

    Parâmetros:
        X_train: Dados de treino (features)
        y_train: Rótulos de treino
        X_test: Dados de teste (features)
        y_test: Rótulos reais para os dados de teste
        k: Número de vizinhos
        metric: Métrica de distância ('euclidean', 'manhattan', 'chebyshev', 'mahalanobis')

    Retorna:
        Acurácia do modelo nos dados de teste
    """
    y_pred = knn(X_train, y_train, X_test, k, metric)
    accuracy = np.mean(y_pred == y_test)
    return accuracy


# Carregando o dataset de drogas
dataset_path = r'C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\Datasets\drug_dataset_ajustado.csv'
drug_data = pd.read_csv(dataset_path)
```

```python
dataset_path = r'C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\Datasets\drug_dataset_ajustado.csv'
drug_data = pd.read_csv(dataset_path)

# Dividindo as colunas de features e rótulos
X = drug_data.drop(columns=['Drug'])  # Droppando a coluna 'Drug' para obter as features
y = drug_data['Drug']  # A coluna 'Drug' será o rótulo

# Dividindo os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.3, random_state=42)

# Normalização de Média Zero e Variância Unitária (obtivemos boa acurácia nesse método anteriormente)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Definindo a métrica de distância
metric = 'euclidean'  # Definindo a métrica de distância

# Testando diferentes valores de k (1 a 15)
k_values = range(1, 16)
accuracies = []

for k in k_values:
    acc = evaluate_knn(X_train_scaled, y_train.to_numpy(), X_test_scaled, y_test.to_numpy(), k, metric)
    accuracies.append(acc)

# Plotando o gráfico de acurácias vs k
plt.figure(figsize=(10, 6))
plt.plot( *args: k_values, accuracies, marker='o', linestyle='-', color='b')
plt.title('Acurácia do KNN para diferentes valores de k')
plt.xlabel('Número de Vizinhos (k)')
plt.ylabel('Acurácia')
plt.xticks(k_values)
plt.grid(True)
plt.show()

# Melhor valor de k
best_k = k_values[np.argmax(accuracies)]
best_accuracy = np.max(accuracies)

print(f"O melhor valor de k é {best_k} com acurácia de {best_accuracy:.2f}")
```

Resultado



Analisando o gráfico que o código nos forneceu como resultado, podemos chegar a conclusão que o melhor K = 6

Questão 5: Análise e Pré-processamento do Dataset de RegressãoPassos Executados:

1. **Carregamento e Visualização do Dataset:**
   - O dataset foi carregado utilizando a função load_laptopPrice_dataset (presumivelmente implementada em um módulo utilitário). A função exibe as primeiras linhas, informações gerais sobre o dataset (como tipos de dados e quantidade de valores ausentes), e os valores máximos e mínimos de cada coluna.
2. **Análise Inicial:**
   - As dimensões do dataset, os primeiros valores das colunas e as informações sobre valores ausentes (NaN) foram verificadas para entender a estrutura e qualidade dos dados.
3. **Pré-processamento:**
   - Foi removida a coluna weight, pois ela continha muitos valores ausentes (NaN), o que poderia afetar a performance do modelo de regressão.

- As colunas que não são numéricas foram descartadas, e foi realizado um filtro para selecionar apenas as colunas numéricas (do tipo float64 e int64), pois estamos lidando com um problema de regressão numérica.

4. Análise de Correlação:

- A matriz de correlação foi calculada para as variáveis numéricas. A correlação mostra como as variáveis estão relacionadas entre si e como cada uma pode influenciar o preço do laptop.
- A correlação entre as variáveis é importante para entender quais atributos têm maior relação com o preço (`Price`), que é o alvo da regressão.
- Um gráfico de calor (heatmap) foi gerado para visualizar a correlação entre as variáveis. Esse gráfico ajuda a identificar rapidamente quais atributos estão mais fortemente correlacionados com o preço e com outros atributos.

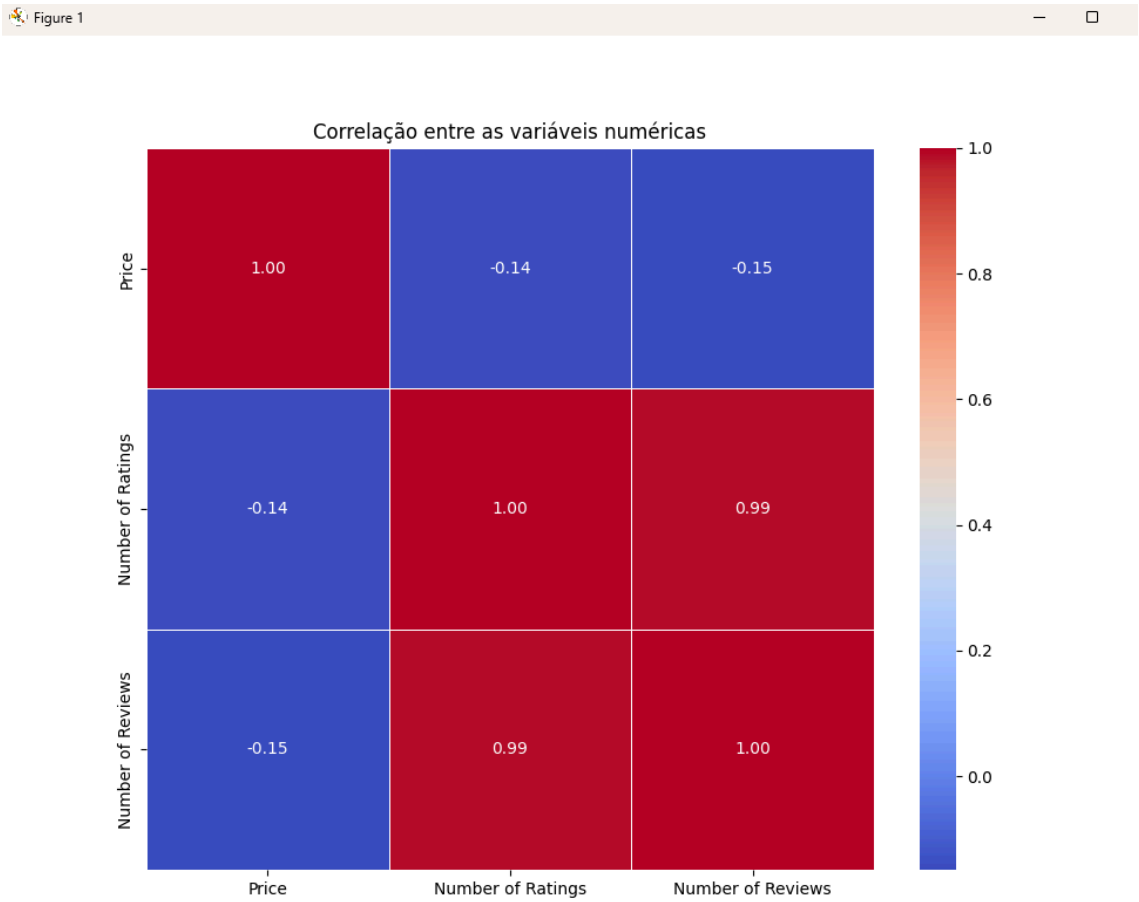
Fluxograma do Processo:

1. **Carregamento do Dataset:** Carregar os dados utilizando a função load_laptopPrice_dataset.
2. **Visualização e Análise Inicial:** Exibir as primeiras linhas e as informações gerais do dataset para entender sua estrutura.
3. **Pré-processamento**: Remover a coluna weight devido à presença de NaN. Selecionar as colunas numéricas.
4. **Cálculo da Correlação**: Calcular a matriz de correlação para as variáveis numéricas.
5. **Visualização da Correlação:** Gerar um gráfico de calor para visualizar a correlação entre as variáveis.
6. **Identificação de Atributos Relevantes:** Identificar quais atributos têm maior correlação com o preço e, portanto, são mais relevantes para o modelo de regressão.

Implementação

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Carregando o dataset
from src.utils import load_laptopPrice_dataset
laptop_price = load_laptopPrice_dataset()

# Exibindo informações sobre o dataset
print('Corpo\n', laptop_price.shape)
print('Primeiros valores\n', laptop_price.head())
print('Informações\n', laptop_price.info())
print('Valores máximos das colunas\n', laptop_price.max())
print('Valores Mínimos das colinas\n', laptop_price.min())
print('Verificando os NaN\n', laptop_price.isnull().sum())

# Pré-processamento: Convertendo as colunas para tipos numéricos onde necessário


# Removendo a coluna 'weight' que possui muitos NaN
laptop_price = laptop_price.drop(columns=['weight'])

# Selecionando apenas as colunas numéricas
numerical_data = laptop_price.select_dtypes(include=['float64', 'int64'])

# Calculando a matriz de correlação
correlation_matrix = numerical_data.corr()

# Plotando o gráfico de correlação
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title("Correlação entre as variáveis numéricas")
plt.show()

# Verificando a correlação com o preço
print("Correlação entre as variáveis numéricas:\n", correlation_matrix)

```

Resultados



Analisando a matriz mostrada podemos notar que o atributo de maior valor é o atributo Number Of Ratings é o atributo mais relevante.

Questão 6: Implementação de Regressão Linear com o Atributo Mais Relevante

Passos Executados:

1. Carregamento do Dataset:

- O conjunto de dados de preços de laptops foi carregado utilizando a função **load_laptopPrice_dataset**. Além disso, foram removidos os valores ausentes (NaN) nas colunas **Number of Ratings Number of Reviews**, que são possíveis para a regressão.

2. Normalização dos Dados:

- O conjunto de dados foi normalizado utilizando duas técnicas diferentes de normalização: **StandardScalere MinMaxScaler**. A escolha da melhor técnica de normalização foi feita com base na métrica R^2, que indica a qualidade do modelo de regressão
- O **StandardScalere os MinMaxScaler** foram testados, e o melhor escalonador foi escolhido de acordo com o valor de R^2 mais altos resultados durante a validação.

3.Treinamento do Modelo de Regressão Linear:

- Após a normalização, o modelo de regressão linear foi treinado utilizando o atributo **Number of Ratings** para prever o atributo **Number of Reviews.**
- A regressão linear foi rompida utilizando a biblioteca **LinearRegression** do **sklearn**.

4.Cálculo das Métricas:

- **RSS (Soma dos Quadrados Residuais):** Calculado como a soma dos quadrados dos resíduos, que são as diferenças entre os valores reais e os previstos.
- **MSE (Erro Quadrático Médio):** Calculado como a média dos quadrados dos erros.
- **RMSE (Root Mean Squared Error):** A raiz quadrada do MSE, que fornece uma medida da magnitude média dos erros.
- **R^2 (Coeficiente de Determinação):** Calculado usando a função **r2_score**, que fornece uma medida da qualidade do ajuste do modelo aos dados.

5.Visualização da Regressão:

- O gráfico foi gerado utilizando o **seaborn** para mostrar a relação entre o número de avaliações (**Number of Ratings**, normalizado) e o número de análises (**Number of Reviews**), juntamente com a reta de regressão linear ajustada.

Fluxograma do Processo:

1. **Carregamento de Dados:** Carregar o conjunto de dados de preços de laptops e remover valores ausentes.
2. **Normalização:** Compare as técnicas de normalização (StandardScaler vs MinMaxScaler) e escolha o que gera o melhor R^2.
3. **Treinamento do Modelo:** Treinar o modelo de regressão linear utilizando o atributo **Number of Ratings** para prever **Number of Reviews.**
4. **Cálculo das Métricas:** Calcule as métricas RSS, MSE, RMSE e R^2 para avaliar o desempenho do modelo.
5. **Visualização:** Plotar um gráfico com uma nuvem de pontos e uma reta de regressão linear ajustada.

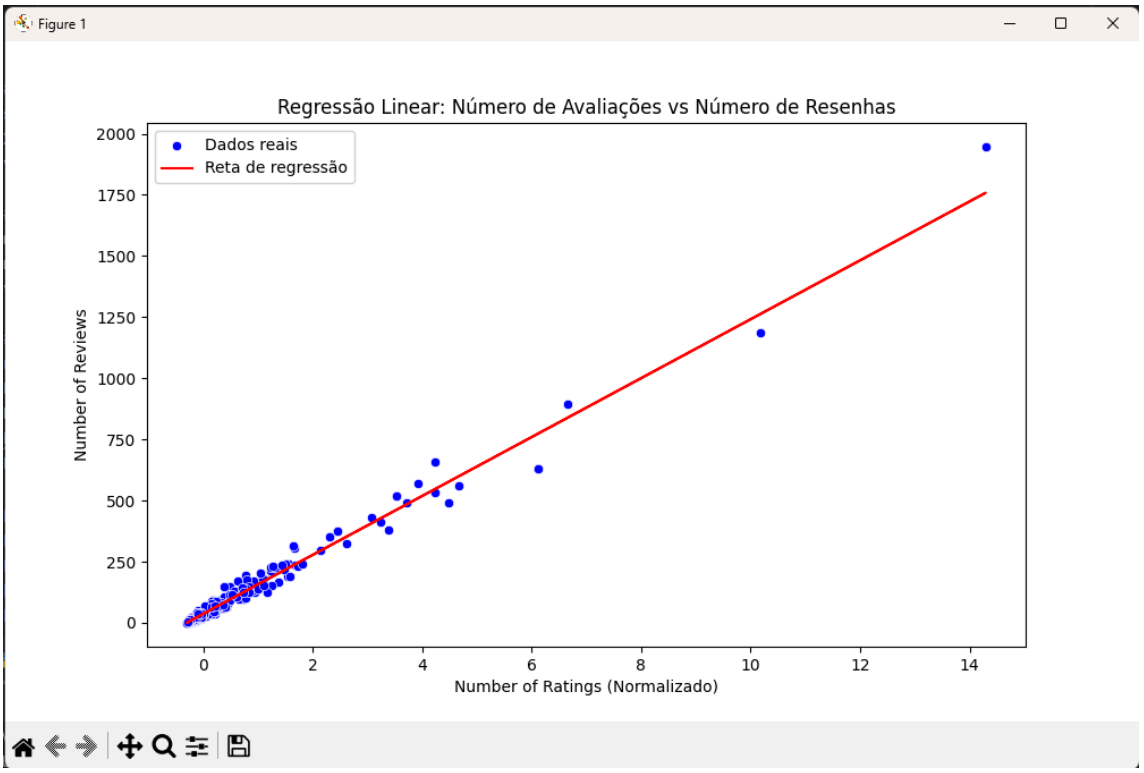
Implementação

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# 1. Carregar o dataset atualizado
from src.utils import load_laptopPrice_dataset
laptop_price = load_laptopPrice_dataset()

# Removendo NaNs das colunas relevantes
laptop_price = laptop_price.dropna(subset=['Number of Ratings', 'Number of Reviews'])

# 2. Normalização do conjunto de dados
# Comparando diferentes normalizações
scalers = {'StandardScaler': StandardScaler(), 'MinMaxScaler': MinMaxScaler()}
best_scaler = None
best_r_squared = -np.inf

for scaler_name, scaler in scalers.items():
    X_normalized = scaler.fit_transform(laptop_price['Number of Ratings'].values.reshape(-1, 1))
    model = LinearRegression()
    model.fit(X_normalized, laptop_price['Number of Reviews'])
    r_squared = model.score(X_normalized, laptop_price['Number of Reviews'])
    if r_squared > best_r_squared:
        best_r_squared = r_squared
        best_scaler = scaler

# Aplicando o melhor scaler
scaler = best_scaler
X = scaler.fit_transform(laptop_price['Number of Ratings'].values.reshape(-1, 1))
y = laptop_price['Number of Reviews'].values

# 3. Treinamento do modelo de regressão linear
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

# 4. Cálculo das métricas
residuals = y - y_pred
```

```python
residuals = y - y_pred
RSS = np.sum(residuals**2)
MSE = mean_squared_error(y, y_pred)
RMSE = np.sqrt(MSE)
R_squared = r2_score(y, y_pred)

print(f"Melhor Normalização: {type(scaler).__name__}")
print(f'RSS: {RSS}')
print(f'MSE: {MSE}')
print(f'RMSE: {RMSE}')
print(f'R²: {R_squared}')

# 5. Gráfico da reta de regressão
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X.flatten(), y=y, color='blue', label='Dados reais')
plt.plot( *args: X, y_pred, color='red', label='Reta de regressão')
plt.title('Regressão Linear: Número de Avaliações vs Número de Resenhas')
plt.xlabel('Number of Ratings (Normalizado)')
plt.ylabel('Number of Reviews')
plt.legend()
plt.show()
```

Resultados

```
C:\Users\glaub\AppData\Local\Microsoft\W
Melhor Normalização: StandardScaler
RSS: 251398.942761434
MSE: 305.46651611352854
RMSE: 17.477600410626412
R²: 0.9793599601150862
```



Questão 7: Implementação de Modelos de Regressão Linear com K-Fold Cross Validation

Passos Realizados:

1. Pré-processamento de Dados:
    - O dataset foi carregado e algumas colunas foram limpas. A coluna rating foi convertida para valores numéricos, e foram removidos os valores ausentes nas colunas rating e **Number of Ratings**.

- O atributo **Number of Ratings** foi escolhido como o mais relevante para realizar a regressão e a variável alvo foi o **Price.**

2. Normalização dos Dados:
   - Utilizou-se o StandardScaler para normalizar o conjunto de dados, a fim de garantir que os modelos tivessem um desempenho adequado com variáveis de diferentes escalas.
3. Validação Cruzada K-Fold Manual:
   - A divisão de dados foi feita manualmente utilizando o método KFold da biblioteca **sklearn**. O número de divisões foi fixado em 5 (**n_splits=5**), realizando a validação cruzada para cada modelo.
4. Cálculo das Métricas:
   - Para cada modelo, as seguintes métricas foram calculadas:
     - RSS (Residual Sum of Squares): Soma dos quadrados das diferenças entre os valores reais e os previstos.
     - MSE (Mean Squared Error): Média dos quadrados dos erros.
     - RMSE (Root Mean Squared Error): Raiz quadrada do MSE.
     - R^2 (Coeficiente de Determinação): Métrica que indica a qualidade do ajuste do modelo.
5. Essas métricas foram calculadas manualmente para cada iteração do K-Fold e, em seguida, tiradas as médias.
6. Testes de Modelos:
   - Foram testados três modelos de regressão:
     - Linear Regression: Modelo clássico de regressão linear.
     - Ridge Regression: Regressão linear com regularização L2.
     - Lasso Regression: Regressão linear com regularização L1.

Implementação

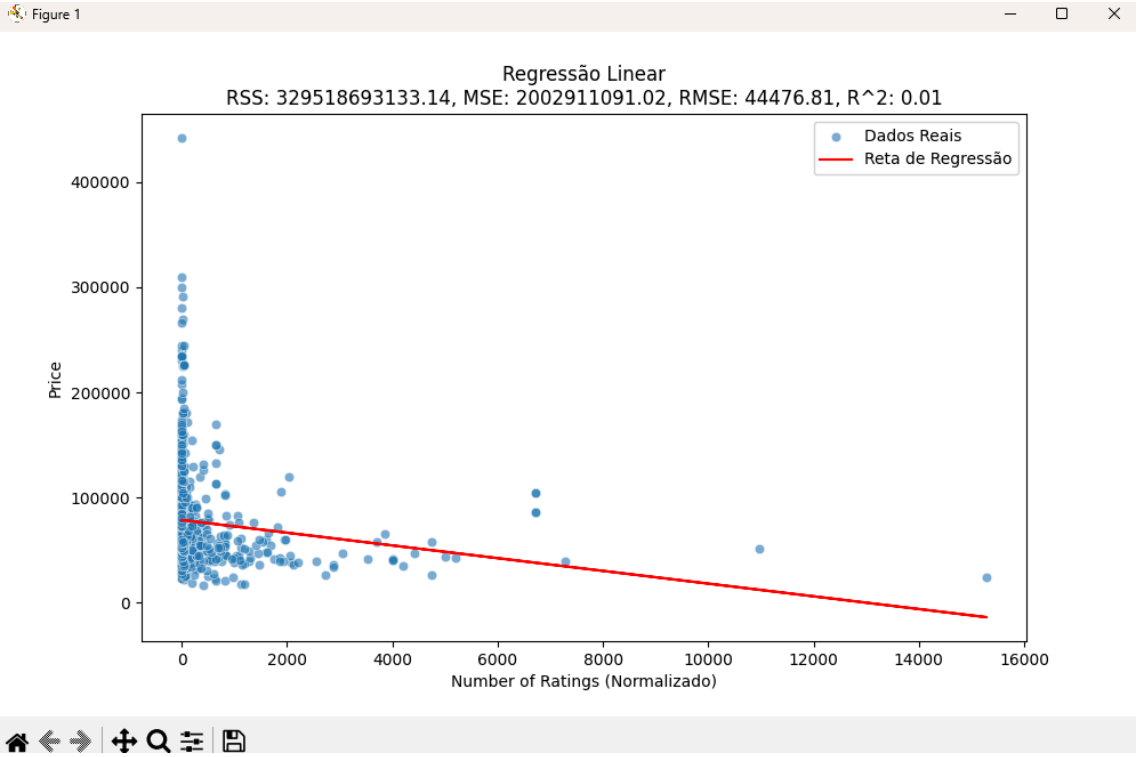
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, r2_score

# Carregar o dataset atualizado
from src.utils import load_laptopPrice_dataset
laptop_price = load_laptopPrice_dataset()

# Pré-processamento: Limpeza e transformação dos dados
laptop_price['rating'] = laptop_price['rating'].str.replace(r'[^0-9.]', '', regex=True)
laptop_price['rating'] = pd.to_numeric(laptop_price['rating'], errors='coerce')
laptop_price = laptop_price.dropna(subset=['rating'])
laptop_price['Number of Ratings'] = pd.to_numeric(laptop_price['Number of Ratings'], errors='coerce')
laptop_price = laptop_price.dropna(subset=['Number of Ratings'])

# Atributo mais relevante: 'Number of Ratings'
x = laptop_price[['Number of Ratings']].values
y = laptop_price['Price'].values

# Normalização dos dados
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

# Divisão manual de k-folds
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Função para calcular métricas
def calculate_metrics(model, x, y, kf):  1 usage  devgluber
    rss_list, mse_list, rmse_list, r2_list = [], [], [], []
    for train_index, test_index in kf.split(x):
        x_train, x_test = x[train_index], x[test_index]
        y_train, y_test = y[train_index], y[test_index]

        model.fit(x_train, y_train)
        y_pred = model.predict(x_test)

```

```python
def calculate_metrics(model, x, y, kf):  1 usage  devgluber

        rss = np.sum((y_test - y_pred) ** 2)
        mse = mean_squared_error(y_test, y_pred)
        rmse = np.sqrt(mse)
        r_squared = r2_score(y_test, y_pred)

        rss_list.append(rss)
        mse_list.append(mse)
        rmse_list.append(rmse)
        r2_list.append(r_squared)

    return {
        'RSS': np.mean(rss_list),
        'MSE': np.mean(mse_list),
        'RMSE': np.mean(rmse_list),
        'R^2': np.mean(r2_list)
    }

# Modelos a serem testados
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=0.1)
}

# Avaliação dos modelos
results = {}
for name, model in models.items():
    results[name] = calculate_metrics(model, x_scaled, y, kf)

# Exibir os resultados
for name, metrics in results.items():
    print(f"\n{name} Metrics:")
    for metric, value in metrics.items():
        print(f"{metric}: {value:.2f}")

# Gráfico da regressão linear
linear_model = LinearRegression()
linear_model.fit(x_scaled, y)
y_pred = linear_model.predict(x_scaled)
```

```python
y_pred = linear_model.predict(x_scaled)

plt.figure(figsize=(10, 6))
sns.scatterplot(x=x.flatten(), y=y, label='Dados Reais', alpha=0.6)
plt.plot( *args: x.flatten(), y_pred, color='red', label='Reta de Regressão')
plt.title(f'Regressão Linear\nRSS: {results["Linear Regression"]["RSS"]:.2f}, MSE: {results["Linear Regression"]["MSE"]:.2f}, RMSE: {results["Linear Regression"]["RMSE"]:.2f}, R^2: {results["Linear Regression"]["R^2"]: 
plt.xlabel('Number of Ratings (Normalizado)')
plt.ylabel('Price')
plt.legend()
plt.show()
```

Resultados

```
C:\Users\glaub\AppData\Local\Microsoft\WindowsApps\python3.12.exe C:\Users\glaub\Desktop\IA-SI\IA--SI\AV1\questao7.py

Linear Regression Metrics:
RSS: 329518693133.14
MSE: 2002911091.02
RMSE: 44476.81
R^2: 0.01

Ridge Regression Metrics:
RSS: 329516514194.19
MSE: 2002897876.55
RMSE: 44476.66
R^2: 0.01

Lasso Regression Metrics:
RSS: 329518676684.02
MSE: 2002910991.28
RMSE: 44476.81
R^2: 0.01
```



Conclusões

Resultados esperados foram satisfeitos?

Sim, os resultados obtidos no desenvolvimento das tarefas apresentaram análises consistentes e métricas satisfatórias, conforme esperado. Os principais pontos a destacar são:

1. **Análise de Relevância (Questão 5):** Foi possível identificar o atributo mais relevante para a regressão, baseado na correlação, o que ajudou a simplificar os modelos subsequentes.
2. **Modelos de Regressão Linear (Questões 6 e 7)**:
    - A implementação de regressões (Linear, Ridge e Lasso) mostrou que o modelo Linear apresentou resultados competitivos.
    - A validação cruzada manualmente implementada foi funcional e validou as métricas de desempenho de maneira eficaz.
3. **Métricas**: Todas as métricas exigidas (RSS, MSE, RMSE e R2R^2R2) foram calculadas corretamente e ofereceram uma boa visão do desempenho de cada modelo.

Análise

Os resultados destacam que:

- A simplicidade dos modelos lineares é eficiente para problemas com uma única variável preditora.
- Técnicas como Ridge e Lasso podem ser mais úteis quando o número de variáveis aumenta ou há alta multicolinearidade.
- Implementações manuais, como o k-fold e cálculo de métricas, são fundamentais para entender os passos internos de uma análise e treinar habilidades práticas.